

Αλέξανδρος Γκιάφης sdi2200284

Άσκηση 2.2

Όνομα υπολογιστικού συστήματος: linux07 της σχολής

Μοντέλο επεξεργαστή: Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz

Αριθμός πυρήνων: 4

Threads ανα πυρήνα: 1

Έκδοση λειτουργικού συστήματος: Ubuntu 20.04.6 LTS

Έκδοση μεταγλωττιστή: gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0

Ακολουθεί η περιγραφή της υλοποίησης της άσκησης 2.2 χωρίς την ενσωμάτωση κώδικα:

Περιγραφή της Υλοποίησης στην Άσκηση 2.2

Στην άσκηση 2.2, το ζητούμενο ήταν η ανάπτυξη ενός προγράμματος που επιλύει ένα άνω τριγωνικό σύστημα γραμμικών εξισώσεων $A \cdot x = b$ με τη μέθοδο της "πίσω αντικατάστασης". Η είσοδος του προγράμματος περιλαμβάνει τις παραμέτρους:

- **matrix_dimension**: Διαστάσεις του πίνακα A,
- **algorithm_type**: Επιλογή του τύπου αλγορίθμου (σειριακός ή παράλληλος),
- **method_type**: Επιλογή μεθόδου (row ή column),
- **num_of_threads**: Ο αριθμός των νημάτων που θα χρησιμοποιηθούν.

Στην υλοποίηση αυτής της άσκησης, αναπτύχθηκαν δύο βασικοί αλγόριθμοι: ο σειριακός και ο παράλληλος για τις μεθόδους **row-wise** και **column-wise**.

Row-wise Αλγόριθμος (Σειριακός και Παράλληλος)

Σειριακή Υλοποίηση

Στη σειριακή υλοποίηση της μεθόδου **row-wise**, ο αλγόριθμος επεξεργάζεται τις γραμμές του πίνακα από κάτω προς τα πάνω. Η διαδικασία περιλαμβάνει την εκχώρηση στις μεταβλητές `x[row]` τις τιμές από το διάνυσμα `b[row]`, και στη συνέχεια την αφαίρεση των συμβολών από τις υπόλοιπες γραμμές, πριν διαιρεθεί το αποτέλεσμα με την αντίστοιχη διαγώνιο τιμή `A[row][row]`.

Παράλληλη Υλοποίηση

Στην παράλληλη υλοποίηση της μεθόδου **row-wise**, η παραλληλοποίηση μπορεί να εφαρμοστεί μόνο στην εσωτερική επανάληψη, καθώς η εξωτερική επανάληψη παρουσιάζει εξαρτήσεις που πρέπει να τηρηθούν. Συγκεκριμένα, δεν μπορούμε να περάσουμε στην επόμενη γραμμή πριν ολοκληρωθεί η επεξεργασία της τρέχουσας, καθώς η επόμενη γραμμή εξαρτάται από την τιμή του `x[row]`, η οποία υπολογίζεται στη συγκεκριμένη γραμμή.

Για να επιτύχω την παραλληλοποίηση:

- Δημιουργώ μια ομάδα από **νήματα (threads)**.

- Χρησιμοποιώ μηχανισμούς **συγχρονισμού** για να διασφαλίσω τη σωστή εκτέλεση του αλγορίθμου από όλα τα νήματα.

Στη διαδικασία υπολογισμού της τιμής του $x[\text{row}]$:

- Εφαρμόζω **reduction** για να συνδυαστούν τα μερικά αποτελέσματα που υπολογίζουν τα νήματα.
- Το **single block** προσθέτει implicit **barrier** έτσι ώστε τα νήματα να μην προχωρήσουν στην επόμενη γραμμή πριν υπολογιστεί η τιμή του $x[\text{row}]$.

Δημιούργησα την μεταβλητή x_i καθώς το MPI δεν επιτρέπει να βάλω το $x[\text{row}]$ για reduction.

Column-wise Αλγόριθμος (Σειριακός και Παράλληλος)

Σειριακή Υλοποίηση

Η σειριακή υλοποίηση της μεθόδου **column-wise** επεξεργάζεται πρώτα το διάνυσμα b , και στη συνέχεια προσπελαύνει τις στήλες του πίνακα από πίσω προς τα εμπρός. Σε κάθε στήλη, υπολογίζεται η τιμή του $x[\text{col}]$ και αφαιρούνται οι συνεισφορές από τις γραμμές που βρίσκονται πάνω από την τρέχουσα στήλη.

Παράλληλη Υλοποίηση

Στον συγκεκριμένο αλγόριθμο, μπορώ να παραλληλοποιήσω τον πρώτο βρόχο που αρχικοποιεί τις τιμές $x[i]$ σε $b[i]$. Στη συνέχεια, στην εμφωλευμένη επανάληψη, η παραλληλοποίηση είναι δυνατή μόνο για τον εσωτερικό βρόχο, λόγω των εξαρτήσεων που υπάρχουν.

Συγκεκριμένα:

- Για να προχωρήσεις στην επόμενη στήλη και να υπολογίσεις την τιμή $x[\text{column}]$ μέσω της διαίρεσης, απαιτείται να έχουν ολοκληρωθεί οι υπολογισμοί που εξαρτώνται από προηγούμενες επαναλήψεις του εξωτερικού βρόχου. Αυτοί οι υπολογισμοί δεν θα είναι εγγυημένα ολοκληρωμένοι αν ο εξωτερικός βρόχος παραλληλοποιηθεί χωρίς προσεκτικό χειρισμό.

Η παραλληλοποίηση του εσωτερικού βρόχου ήταν πιο απλή σε σύγκριση με τον row-wise αλγόριθμο, καθώς σε αυτή την περίπτωση δεν απαιτείται χρήση **reduction** ή πρόσθετων μηχανισμών συγχρονισμού.

Δοκίμασα να παραλληλοποιήσω και τον εξωτερικό βρόχο, αλλά δεν κατάφερα να δημιουργήσω μια αποδοτική υλοποίηση. Για παράδειγμα, στον row-wise αλγόριθμο σκέφτηκα να παραλληλοποιήσω τον εξωτερικό βρόχο χρησιμοποιώντας locks, ώστε κάθε νήμα να σταματά αν η τιμή $x[i]$ δεν είχε ακόμα υπολογιστεί. Ωστόσο, αυτή η προσέγγιση αποδείχθηκε αναποτελεσματική, καθώς η παραλληλία ήταν τελικά πολύ περιορισμένη, ενώ μεγάλο μέρος του χρόνου σπαταλιόταν σε αναμονή. Ακόμα και με fine-grained scheduling, το πρόβλημα παρέμενε, και έτσι εγκατέλειψα την ιδέα.

Δοκιμές

Οι δοκιμές του προγράμματος πραγματοποιήθηκαν σε δύο φάσεις. Στην πρώτη φάση, ο στόχος ήταν να επιβεβαιώσω ότι τα αποτελέσματα του προγράμματος είναι σωστά, καθώς οποιαδήποτε επιπλέον ανάλυση θα ήταν άσκοπη αν τα αποτελέσματα ήταν λανθασμένα.

Για να το επιτύχω αυτό, χρησιμοποίησα την εντολή `make test`, η οποία μεταγλωττίζει το πρόγραμμα ενεργοποιώντας την παράμετρο `TEST_MODE`. Στο πλαίσιο αυτής της διαδικασίας, πρόσθεσα συγκεκριμένες γραμμές κώδικα μεταξύ των εντολών `#ifdef TEST_MODE` σε διάφορα σημεία του προγράμματος. Ο σειριακός αλγόριθμος χρησιμοποιείται για τον υπολογισμό της τιμής `expected_x`, για τον οποίο είμαι βέβαιος ότι είναι σωστός. Στη συνέχεια, ο υπολογισμός της τιμής `x` γίνεται με τον αλγόριθμο που ζητήθηκε, βασισμένο στις παραμέτρους του προγράμματος. Πριν από την εκτύπωση των χρόνων, το πρόγραμμα ελέγχει ότι το αποτέλεσμα είναι το ίδιο με το `expected_x` με την επιθυμητή ακρίβεια. Εάν η τιμή δεν ταιριάζει, εκτυπώνει τις δύο τιμές ώστε να διευκολυνθεί το debugging. Μέσω αυτής της διαδικασίας, επιβεβαίωσα ότι οι παράλληλοι αλγόριθμοι λειτουργούν σωστά.

Στη συνέχεια, για την εκτέλεση των δοκιμών και τη συλλογή των χρόνων, μεταγλώττισα το πρόγραμμα χωρίς να ορίσω το `TEST_MODE`, καθώς ο επιπλέον έλεγχος καταναλώνει χρόνο και θα καθυστερούσε την ολοκλήρωση των δοκιμών. Στη συνέχεια, εκτέλεσα το script `collect_data`, το οποίο τρέχει τις δοκιμές για διαφορετικές διαστάσεις (64, 1024, 4096, 8192, 16384) και αριθμούς νημάτων (1, 2, 4, 8, 16), για κάθε συνδυασμό εκτελώντας 4 φορές τα τεστ και υπολογίζοντας τους μέσους χρόνους εκτέλεσης για τις row-wise και column-wise εκτελέσεις. Τα δεδομένα αποθηκεύονται σε αρχεία CSV στους εξής φακέλους:

- `collected_data_for_64_size.csv`
- `collected_data_for_1024_size.csv`
- ...

Τα δεδομένα είναι αποθηκευμένα στην παρακάτω μορφή:

```
dimension,threads,row_time,column_time
1024,1,.00341427326225000000,.00569576025000000000
1024,2,.00490552186950000000,.00642234087000000000
1024,4,.00426799058925000000,.00518172979375000000
1024,8,.04375833272925000000,.05960929393775000000
1024,16,.11864733695975000000,.12123376131050000000
```

Τέλος, για την οπτικοποίηση των δεδομένων, χρησιμοποίησα το script `makegraph.py`, το οποίο δημιουργεί γραφήματα από τα δεδομένα που συλλέχθηκαν.

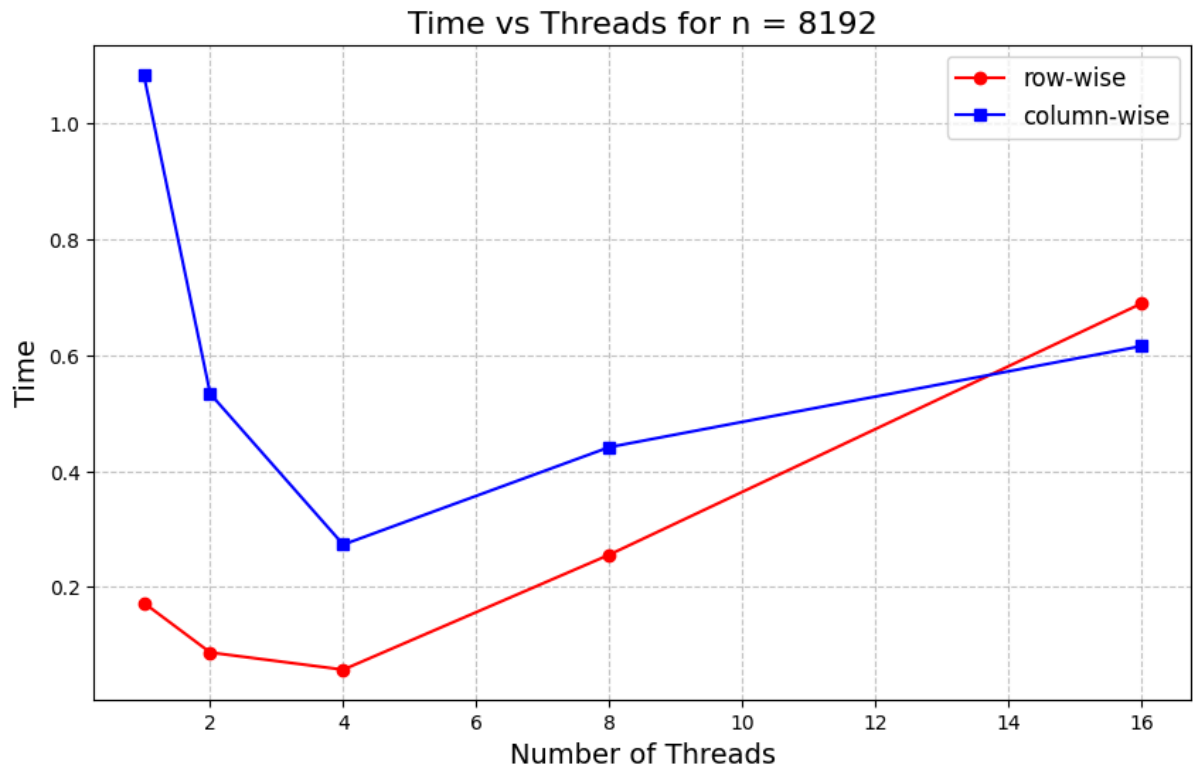
Πριν τρέξεις τα τεστ, μπορείς να καθορίσεις τις παραμέτρους του scheduling εκτελώντας την εντολή `export OMP_SCHEDULE="type, chunk_size"`. Με αυτόν τον τρόπο, συλλέχθηκαν δεδομένα για διαφορετικές επιλογές scheduling.

Τα αποτελέσματα:

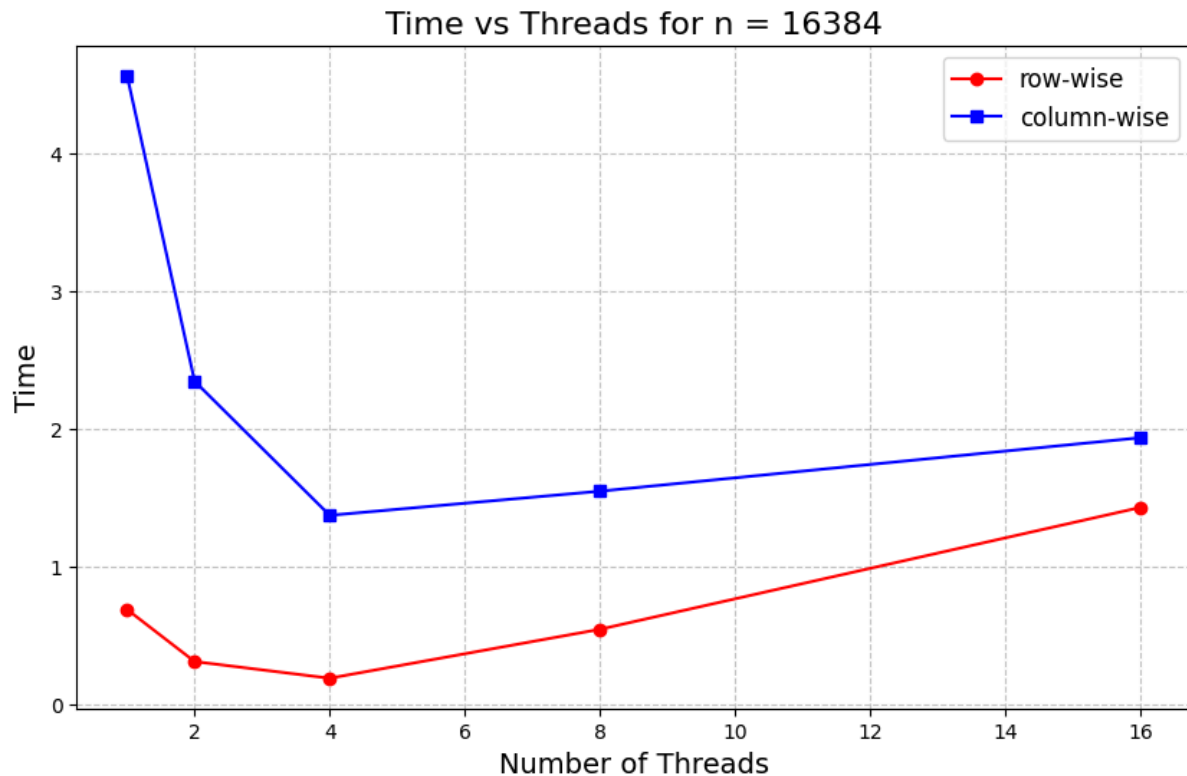
Για `OMP_SCHEDULE="static"`

Βλέπουμε χρόνο εκτέλεσης ανάλογα τον αριθμό από threads για row-wise και column-wise αλγόριθμους:

Για $n = 8192$:



Για $n = 16384$:



Ο αλγόριθμος column-wise φαίνεται σε κάθε περίπτωση να είναι πιο αργός σε σχέση με τον row-wise αλγόριθμο. Κατά την άποψή μου, αυτό οφείλεται στο γεγονός ότι ο τρόπος προσπέλασης του πίνακα A από τον αλγόριθμο column-wise δεν είναι cache-friendly, σε αντίθεση με τον αλγόριθμο row-wise. Συγκεκριμένα, ο column-wise αλγόριθμος προκαλεί περισσότερα cache misses, καθώς η προσπέλαση των στοιχείων του πίνακα γίνεται σε μη συνεχείς θέσεις μνήμης. Αντιθέτως, στον row-wise αλγόριθμο, τα δεδομένα προσπελαύνονται σειριακά για κάθε γραμμή, γεγονός που επιτρέπει την ανάκτηση πολλών δεδομένων στην μνήμη με ένα μόνο cache miss. Αυτό οδηγεί σε καλύτερη απόδοση στον αλγόριθμο row-wise.

Όσον αφορά τα αποτελέσματα καθώς αυξάνεται ο αριθμός των νημάτων, παρατηρούμε βελτίωση στους χρόνους εκτέλεσης καθώς τα νήματα αυξάνονται, μέχρι να φτάσουμε τα 4 νήματα. Από εκεί και πέρα, όταν ο αριθμός των νημάτων ξεπερνά τα 4, ο χρόνος εκτέλεσης αρχίζει να αυξάνεται. Αν και θα μπορούσε κάποιος να αναμένει ότι ο χρόνος θα παρέμενε σταθερός κοντά στην τιμή που είχε για 4 νήματα, αυτό δεν συμβαίνει. Ο λόγος γι' αυτό είναι ότι με την αύξηση των νημάτων, αυξάνεται και η συνολική δουλειά που απαιτείται για την εκτέλεση του αλγορίθμου.

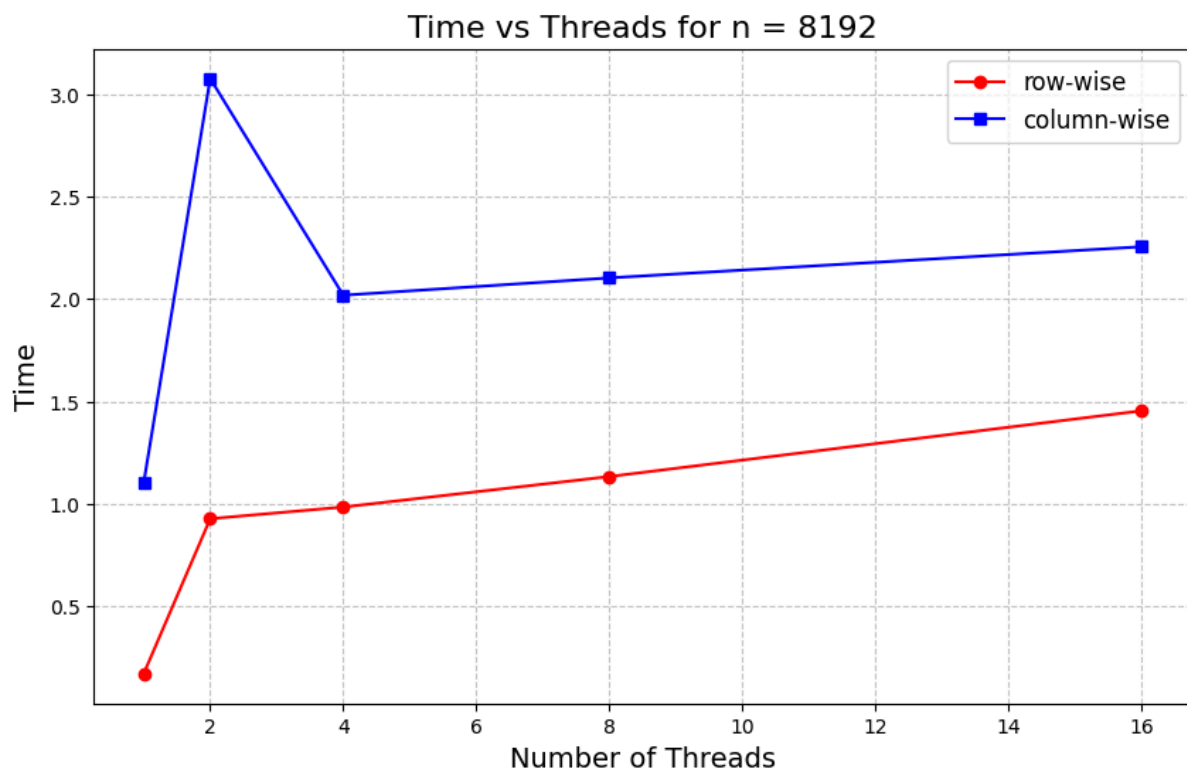
Για παράδειγμα, στον **row-wise αλγόριθμο**, υπάρχει η ανάγκη για **reduction**: κάθε νήμα, εκτός από την εκτέλεση των iterations του, πρέπει να συνδυάζει τα αποτελέσματά του στην κοινή μεταβλητή `xi`. Αυτό απαιτεί συγχρονισμό, ώστε τα νήματα να μην γράφουν στην ίδια μεταβλητή παράλληλα, κάτι που προσθέτει overhead και μπορεί να αυξήσει τον χρόνο εκτέλεσης.

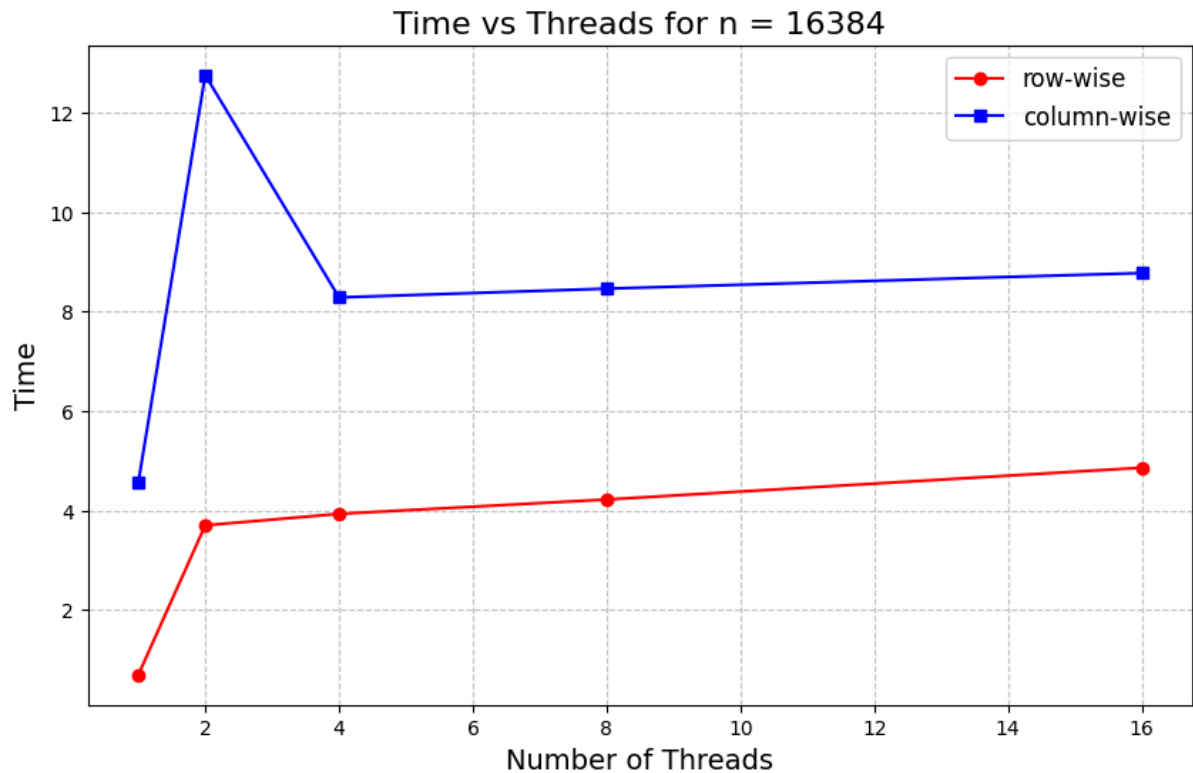
Στον **column-wise αλγόριθμο**, αν και δεν απαιτείται reduction, εξακολουθούμε να παρατηρούμε μικρή αύξηση στους χρόνους. Αυτό οφείλεται στο ότι εξακολουθεί να υπάρχει

το implicit barrier μετά το εσωτερικό βρόχο το οποίο δημιουργεί κάποιο overhead. Αλλά το overhead είναι λιγότερο από τον **row-wise** αλγόριθμο για αυτό και ο χρόνος γίνεται μέχρι και μικρότερος του **row-wise** για 16 νήματα στην πρώτη εικόνα. Και γενικότερα φαίνεται από την κλήση της αύξησης ότι για **column-wise** η αύξηση δεν είναι τόσο απότομη.

Πειραματίστηκα με διάφορες επιλογές για το scheduling. Η παραλληλοποίηση που έχουμε εφαρμόσει διασφαλίζει ότι κάθε iteration έχει περίπου τον ίδιο φόρτο εργασίας, επομένως δεν προκύπτει το πρόβλημα όπου κάποιο νήμα μπορεί να επεξεργάζεται πιο βαριά iteration από κάποιο άλλο. Παρόλα αυτά, υπάρχουν κάποια ενδιαφέρον φαινόμενα που μπορούμε να παρατηρήσουμε.

Για OMP_SCHEDULE = “dynamic”



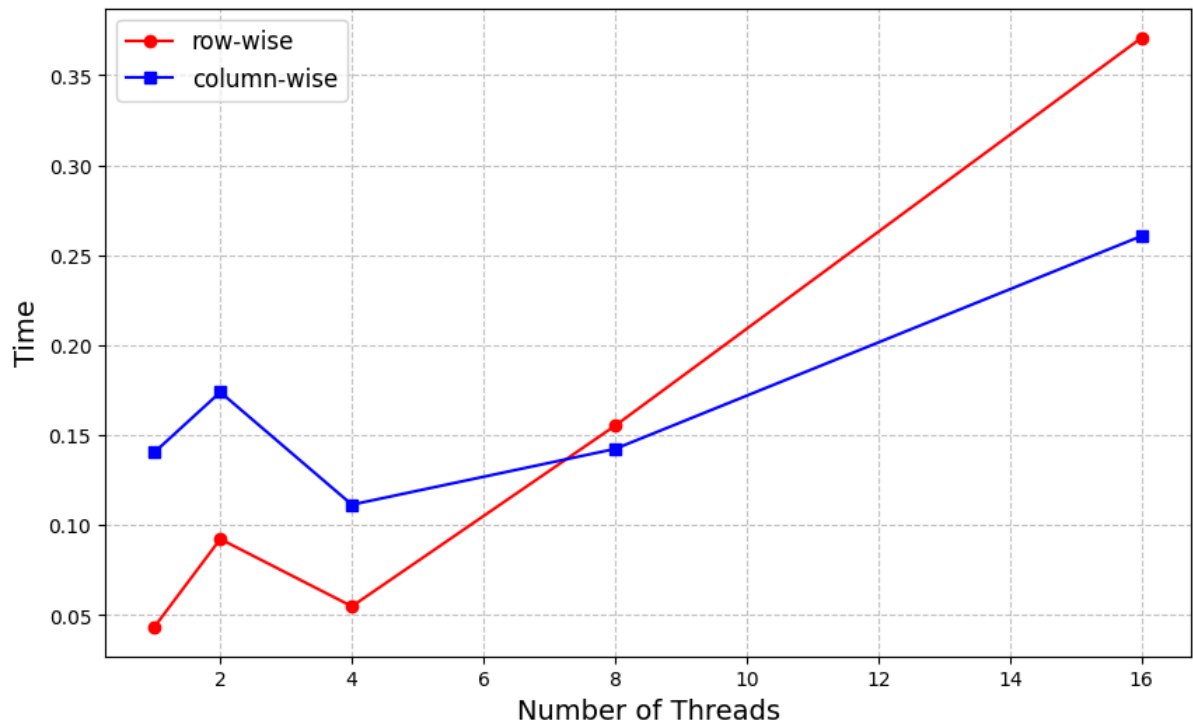


Όταν χρησιμοποιούμε το **dynamic** scheduling στην OpenMP, η κατανομή των επαναλήψεων γίνεται δυναμικά κατά την εκτέλεση, με αποτέλεσμα τα νήματα να αναλαμβάνουν τμήματα εργασίας εν κινήσει. Το κύριο πρόβλημα εδώ είναι το **overhead** από την ανακατανομή εργασιών, το οποίο προκαλεί καθυστερήσεις και μπορεί να αντισταθμίσει τα πλεονεκτήματα της παραλληλίας.

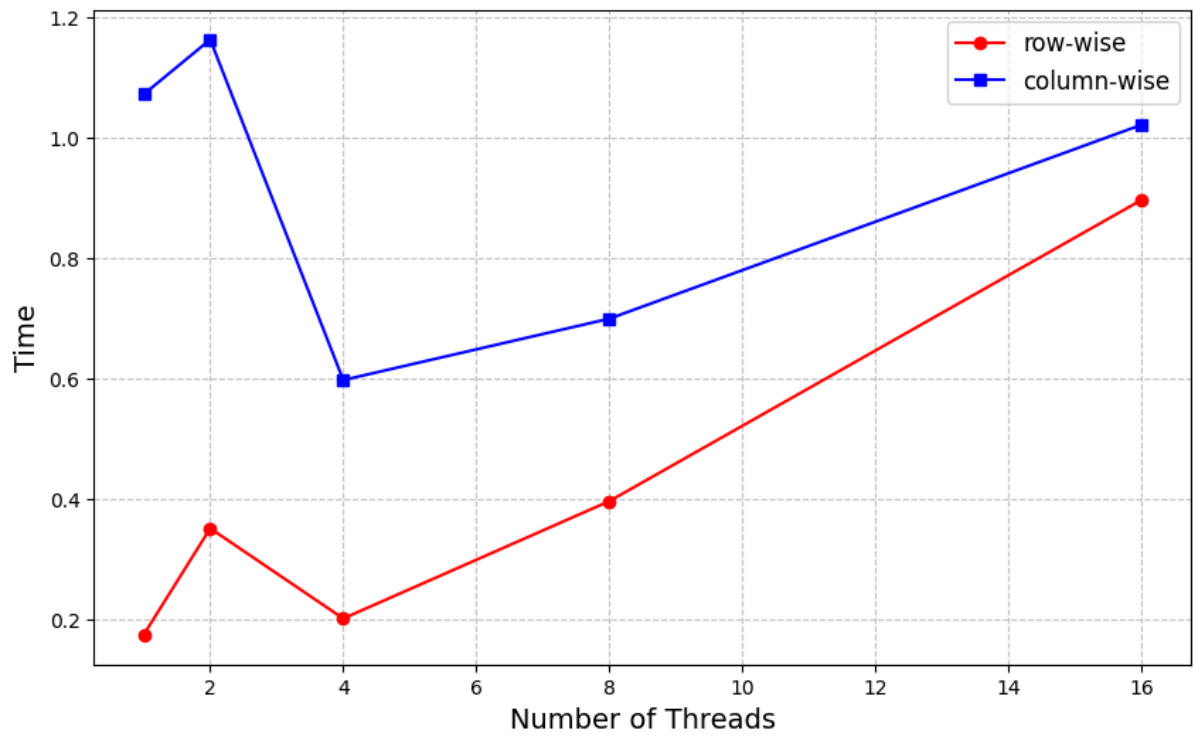
Για 4 νήματα, ο χρόνος εκτέλεσης είναι καλύτερος σε σχέση με 2 νήματα, αλλά χειρότερος από ό,τι με 1 νήμα. Αυτό συμβαίνει διότι, παρόλο που έχουμε περισσότερη παραλληλοποίηση από τον 2, το overhead παραμένει, και αυτό καθυστερεί τη διαδικασία. Όταν ξεπερνάμε τα 4 νήματα, οι χρόνοι επιδεινώνονται, καθώς ο υπολογιστής έχει μόνο 4 πυρήνες και δεν μπορούμε να εκμεταλλευτούμε την παραλληλία, ενώ το overhead από τα επιπλέον νήματα επιβαρύνει την απόδοση.

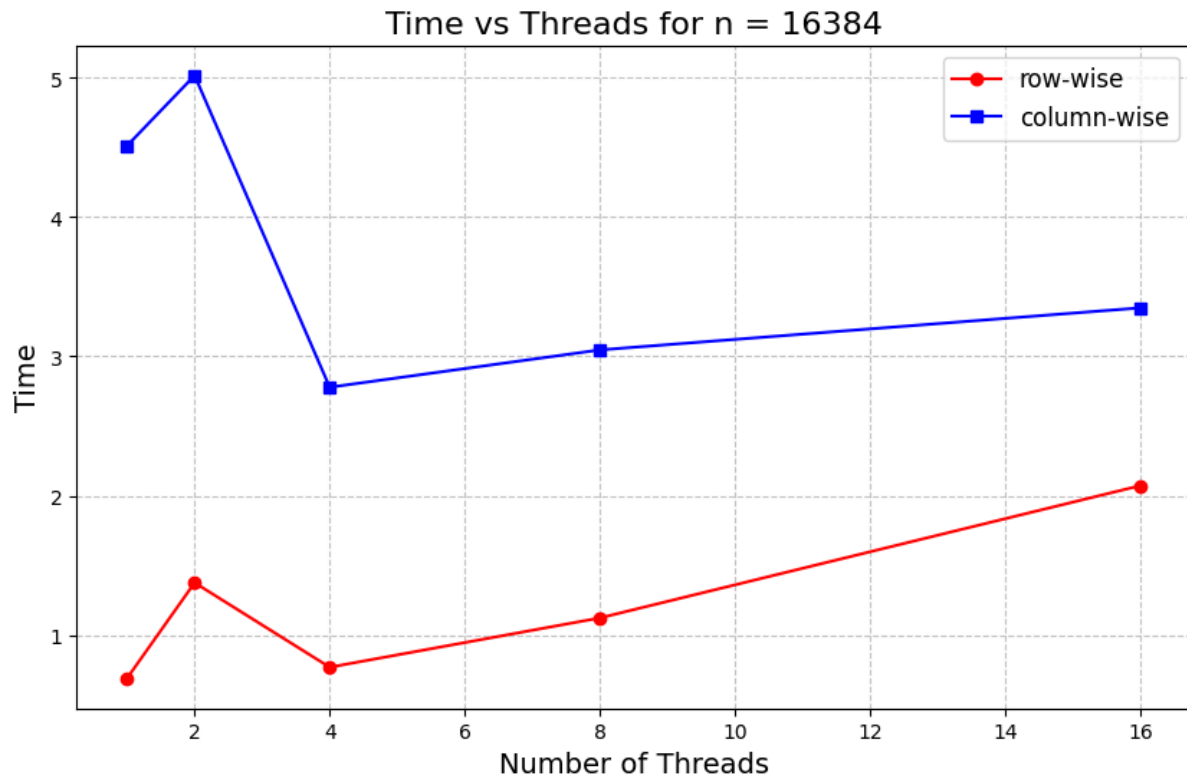
Για `OMP_SCHEDULE="static, 1"`

Time vs Threads for n = 4096



Time vs Threads for n = 8192





Παρατηρούμε μια παράξενη συμπεριφορά: για δύο νήματα, το πρόγραμμα φαίνεται να είναι πιο αργό από τη σειριακή εκτέλεση. Πιστεύω ότι αυτό οφείλεται στο γεγονός ότι τα νήματα δεν μοιράζονται τις ίδιες L1 caches. Με το απλό static scheduling, τα νήματα θα μοιράζονταν τη μνήμη σε συνεχόμενα blocks, ενώ τώρα τα blocks είναι πιο αναμειγμένα. Συγκεκριμένα, το πρώτο νήμα επεξεργάζεται το πρώτο iteration, το δεύτερο νήμα το δεύτερο iteration, και το μοτίβο αυτό συνεχίζεται. Ως αποτέλεσμα, τα ίδια blocks μνήμης γράφονται στις caches και των δύο νημάτων, προκαλώντας έτσι cache contention. Με διαφορετικό scheduling, θα μπορούσαν να γράφονται διαφορετικά blocks για κάθε νήμα, μειώνοντας το overhead. Για τέσσερα νήματα, η κατάσταση βελτιώνεται ελαφρώς, πιθανόν επειδή η αύξηση της παραλληλίας υπερκαλύπτει το overhead των cache misses.